

GUOB

TECH DAY

2026

Patrocínio e Apoio

◆ DIAMANTE

Pythian

★ OURO

ORACLE

◆ BRONZE



APOIO OFICIAL





Beyond Manual Caching

How Readysset takes MySQL to the next level

Vinicius Grippa

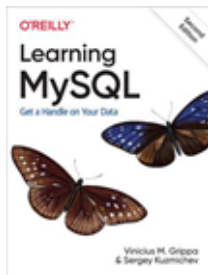
Lead Database Engineer @ Readysset

About Me



SELECT * FROM person WHERE name LIKE 'Vini';

- Lead Database Engineer at **Readysset**
- Ex-Percona working with MySQL and MongoDB
- Co-author of the book Learning MySQL
- Organizer of the MySQL community in Brazil (MUG)



COMUNIDADE
MySQL  **BR**

Agenda



Agenda

01

THE LANDSCAPE

Defining In-Memory Databases and the raw speed vs. budget trade-off.

02

MARKET REALITY

Challenges of scale: DDR5 price volatility and physical capacity caps.

03

THE READYSET ENGINE

Architectural overview of automated SQL result caching and auto-maintenance.

04

CACHING MODES

Shallow (TTL) vs. Deep (Incremental View) caching strategies.

05

Performance

Performance analysis: 60x throughput gains and sub-millisecond latency.

06

Q&A

Open discussion, technical resources, and project roadmap.



WHAT IS AN IN-MEMORY DATABASE?

Core Concept: Data lives natively in system MEMORY rather than experiencing execution bottlenecks on a traditional Disk.

Sub-ms Speed

Completely eliminates standard Disk I/O storage bottlenecks to unlock true "instantaneous" performance vectors.

High Concurrency

Effortlessly scales to process millions of concurrent operational request structures simultaneously.

Simplified Logic

Bypasses complex kernel storage layout translation steps via direct CPU-to-Memory channel communication pathways.

POPULAR INDUSTRY EXAMPLES:

Readysset

Redis & Memcached

SAP HANA

CHALLENGES & MARKET REALITY



Economic Hurdle

High Cost Per GB makes In-Memory storage significantly more expensive than enterprise-grade SSD or HDD solutions.



Price Volatility

Market shifts have led to recent DDR5 price spikes of up to ^{3x}, making infrastructure scaling unpredictable.



Physical Limits

Unlike cloud storage, total data volume is strictly capped by the physical RAM slots available on the server motherboard.

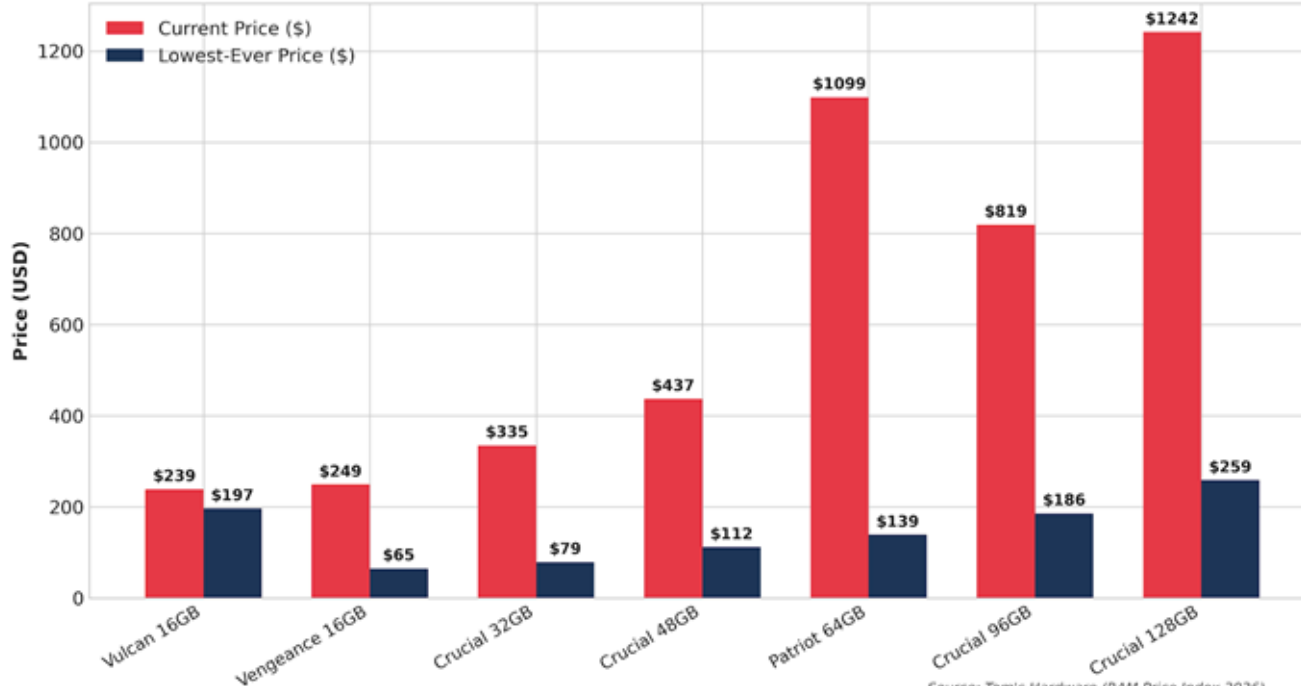
KEY TAKEAWAY

Trading Budget for Raw Operational Speed



Challenges & Market Reality

DDR5 Price Explosion: Market Comparison

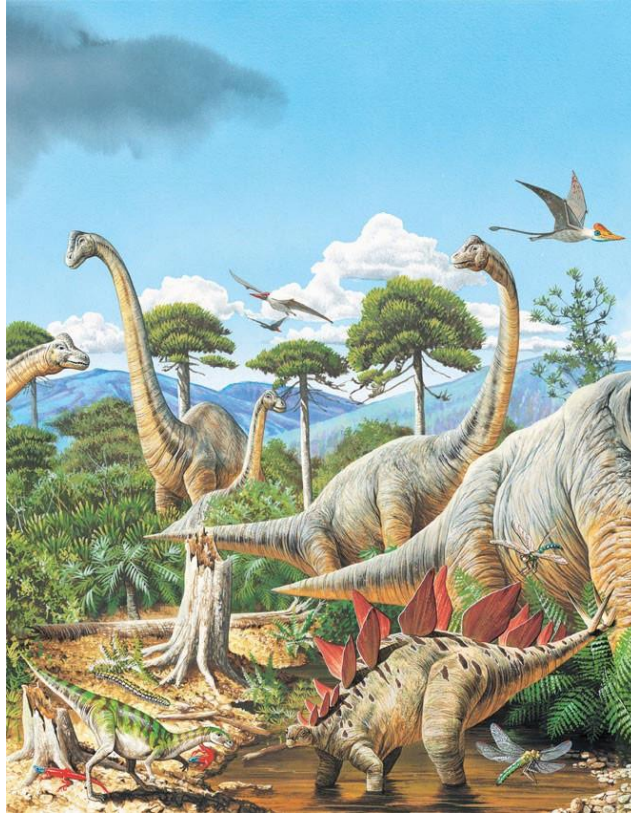


MySQL

Query Cache



MySQL Query Cache – A long time ago...



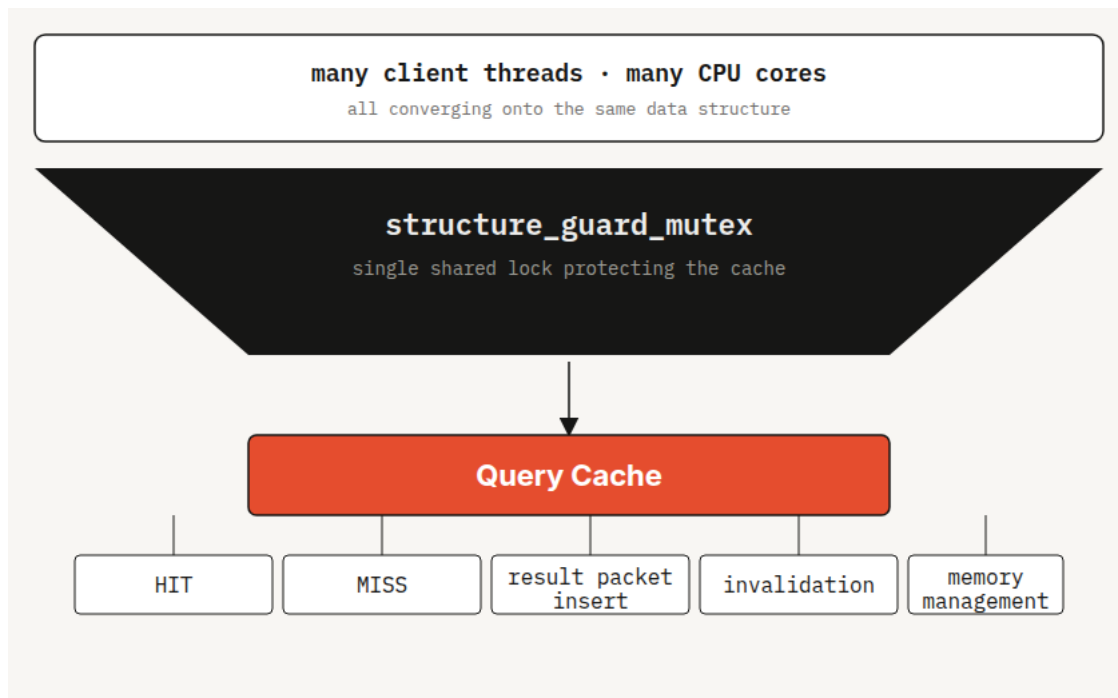
MySQL Query Cache

December 2, 2001. Source tree 4.0.1-alpha.

```
git show 1d26537da53fee228b2f9d974d4806a06b76e33e
Author: monty@hundin.mysql.fi <>
Date: Sun Dec 2 14:34:01 2001 +0200
```

Query cache.
Remove some warnings

MySQL Query Cache



three problems stacked on top

(1) table-wide invalidation · (2) big cache got slower under the same lock · (3) low hit rate by construction

So, was it bad?

MySQL Query Cache

The screenshot shows the MySQL 5.7 Reference Manual page for the Query Cache. The left sidebar contains a navigation menu with sections like 'Documentation Home', 'MySQL 5.7 Reference Manual', 'Preface and Legal Notices', 'General Information', 'Installing and Upgrading MySQL', 'Tutorial', 'MySQL Programs', 'MySQL Server Administration', 'Security', 'Backup and Recovery', 'Optimization', and 'Buffering and Caching'. The main content area is titled '8.10.3 The MySQL Query Cache' and includes sub-sections: '8.10.3.1 How the Query Cache Operates', '8.10.3.2 Query Cache SELECT Options', '8.10.3.3 Query Cache Configuration', and '8.10.3.4 Query Cache Status and Maintenance'. A 'Note' box states: 'The query cache is deprecated as of MySQL 5.7.20, and is removed in MySQL 8.0.' The text explains that the query cache stores the text of a `SELECT` statement and its result, and is shared among sessions. It also mentions that the cache does not return stale data and is flushed when tables are modified. Another 'Note' box states: 'The query cache does not work in an environment where you have multiple `mysqld` servers updating the same `MySQL` tables.' A third 'Note' box states: 'The query cache is used for prepared statements under the conditions described in Section 8.10.3.1, "How the Query Cache Operates".' A red callout box with a black border contains the text: '...running the MySQL benchmark suite on a Linux Alpha x>500 MHz system with 2 GB RAM and a 64 MB query cache.'

dev.mysql.com/doc/refman/5.7/en/query-cache.html?utm_source=chatgpt.com

8.10.3 The MySQL Query Cache

- 8.10.3.1 How the Query Cache Operates
- 8.10.3.2 Query Cache SELECT Options
- 8.10.3.3 Query Cache Configuration
- 8.10.3.4 Query Cache Status and Maintenance

Note

The query cache is deprecated as of MySQL 5.7.20, and is removed in MySQL 8.0.

The query cache stores the text of a `SELECT` statement together with the corresponding result that was sent to the client. If an identical statement is received later, the server retrieves the results from the query cache rather than parsing and executing the statement again. The query cache is shared among sessions, so a result set generated by one client can be sent in response to the same query issued by another client.

The query cache can be useful in an environment where you have tables that do not change very often and for which the server receives many identical queries. This is a typical situation for many Web servers that generate many dynamic pages based on database content.

The query cache does not return stale data. When tables are modified, any relevant entries in the query cache are flushed.

Note

The query cache does not work in an environment where you have multiple `mysqld` servers updating the same `MySQL` tables.

The query cache is used for prepared statements under the conditions described in Section 8.10.3.1, "How the Query Cache Operates".

Note

The query cache is not supported for partitioned tables. The query cache can be enabled for such queries.

Some performance data for the query cache follows. This data was generated on a system with 2 GB RAM and a 64MB query cache.

- If all the queries you are performing are simple (such as selecting a row from a table with one row), but still differ so that the queries cannot be cached, the overhead for having the query cache active is 13%. This could be regarded as the worst case scenario. In real life, queries tend to be much more complicated, so the overhead normally is significantly lower.
- Searches for a single row in a single-row table are 238% faster with the query cache than without it. This can be regarded as close to the minimum speedup to be

“The hard problem is not *storing* results in memory; it is knowing when they are no *longer* valid.”



Readysset

SQL caching engine



A decade of research. One problem.

origin

A decade of MIT research behind it

The **Noria** paper (OSDI '18) describes the **partial dataflow** model Readysset is built on — from MIT PDOS, a systems group focused on parallel and distributed data.

mission

**"There are only two hard things in Computer Science:
cache invalidation and naming things."**

— Phil Karlton

Readysset solves the first. You're on your own for the second.

two modes

Shallow Caching

TTL-based result cache. Lightweight setup, broad coverage.

Deep Caching

Incremental dataflow via CDC/IVM — always consistent.

paper — OSDI '18

Noria: dynamic, partially stateful data-flow for high performance web applications

Jon Gjengset - MIT PDOS · 2018



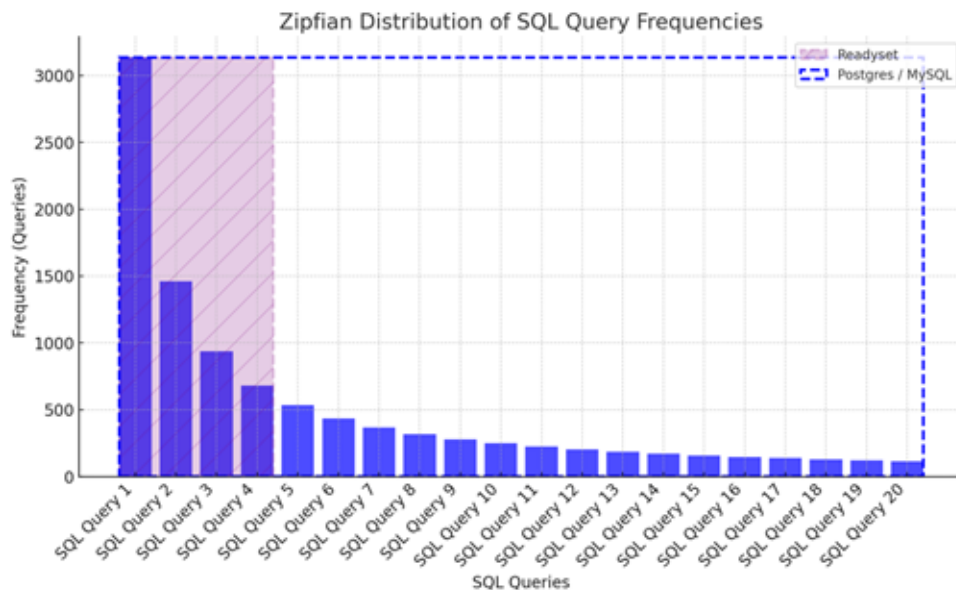
pdos.csail.mit.edu/papers/noria/osdi18.pdf

open source

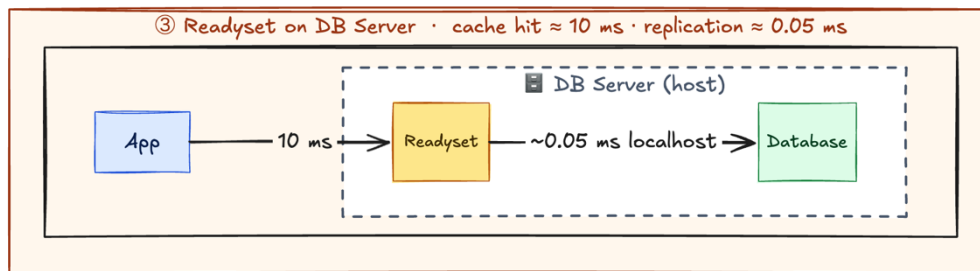
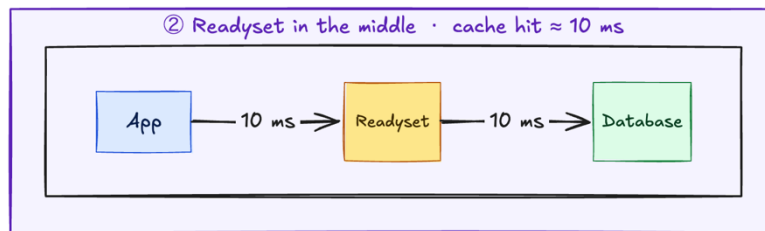
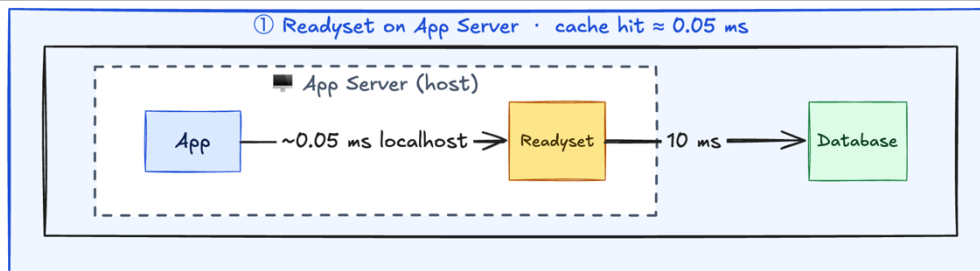
github.com/readyssettech/readysset

Readysset

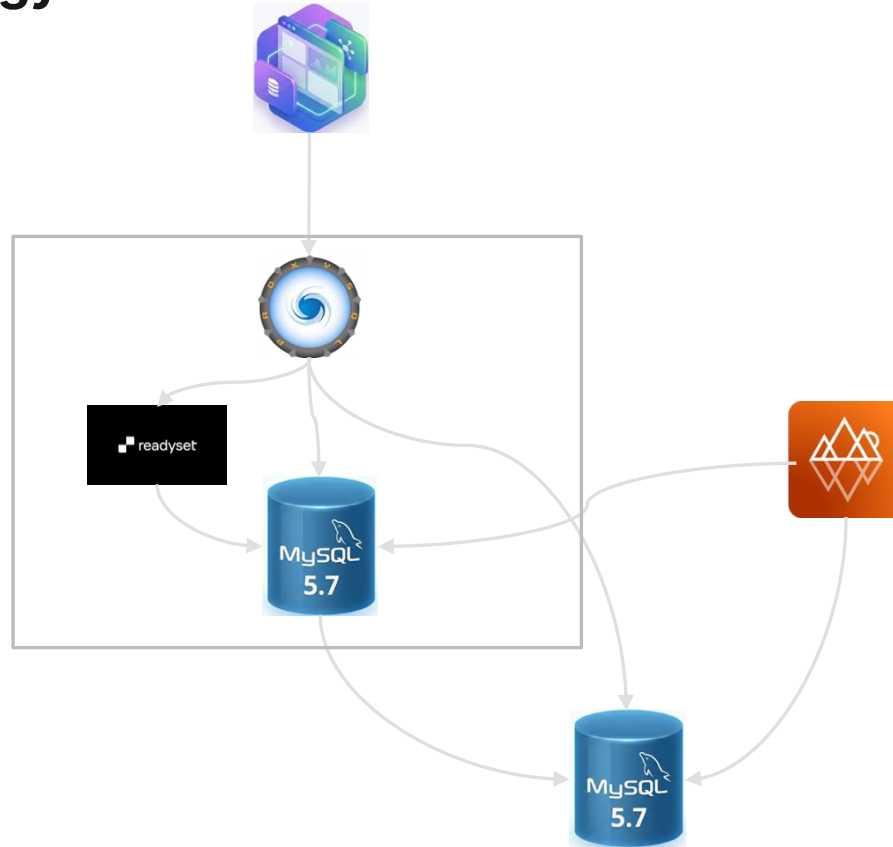
- Queries often follow a Zipfian distribution
- Read heavy workloads. Infrequent changes. Eventual consistency is okay.



Architecture Examples



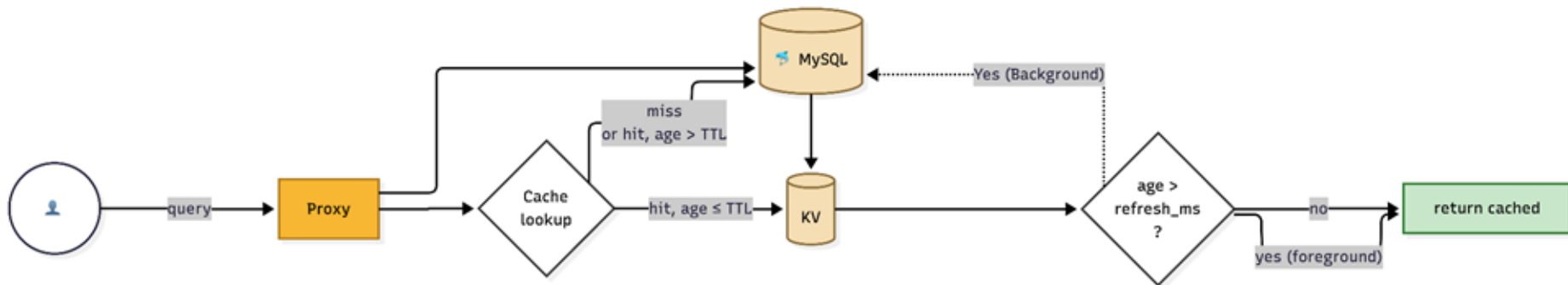
Lemit Topology



Shallow Caching



Shallow Caching



```
CREATE SHALLOW CACHE POLICY TTL 10 SECONDS REFRESH 5 SECONDS  
COALESCE 5 SECONDS FROM SELECT a,b FROM ...;
```

Shallow Caching

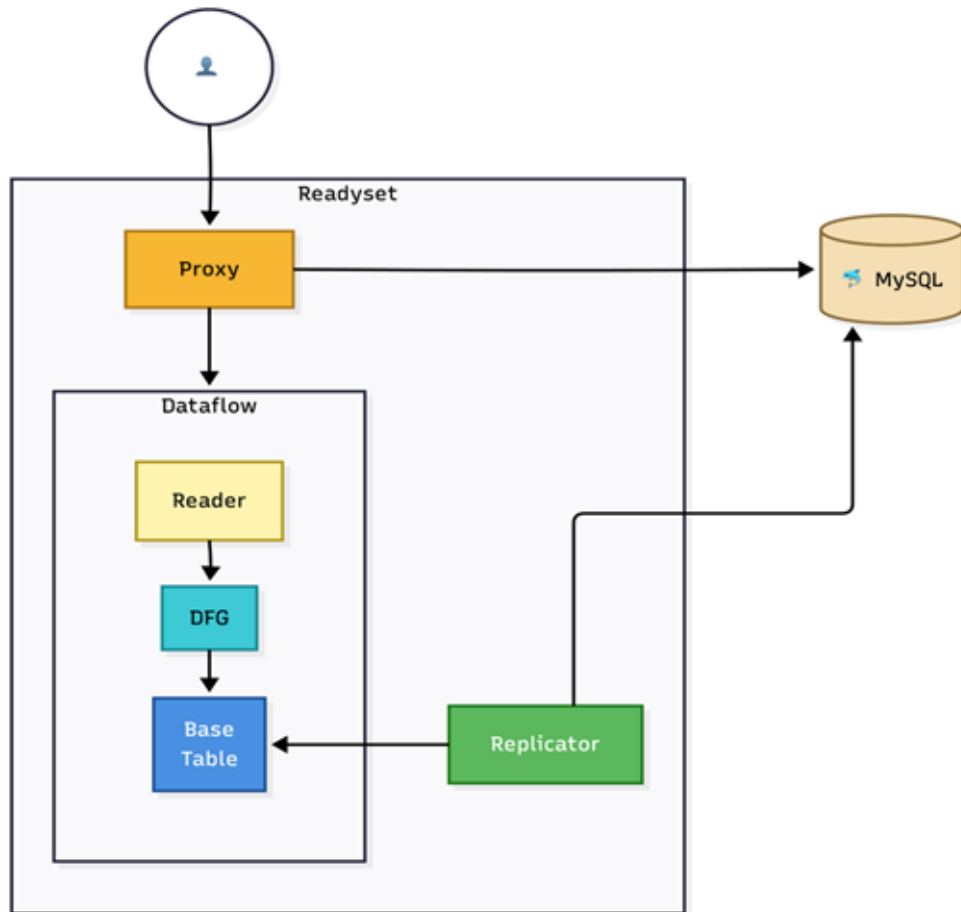
- **TTL-based query result cache:** Shallow caching stores query results in memory for a configured time window, then refreshes when entries become stale.
- **No dataflow materialization required:** It does not build a full incremental view graph, which makes setup lighter and faster.
- **Broader query compatibility:** It can cache many queries that deep/dataflow caching may not support yet.
- **Freshness is policy-driven:** Data is refreshed by TTL and optional refresh settings (on-demand or scheduled), not by per-row CDC dependency tracking.
- **Good fit for pragmatic acceleration:** Useful when you need quick read latency improvements with simpler operational overhead, while accepting TTL-style freshness tradeoffs.

Deep Caching

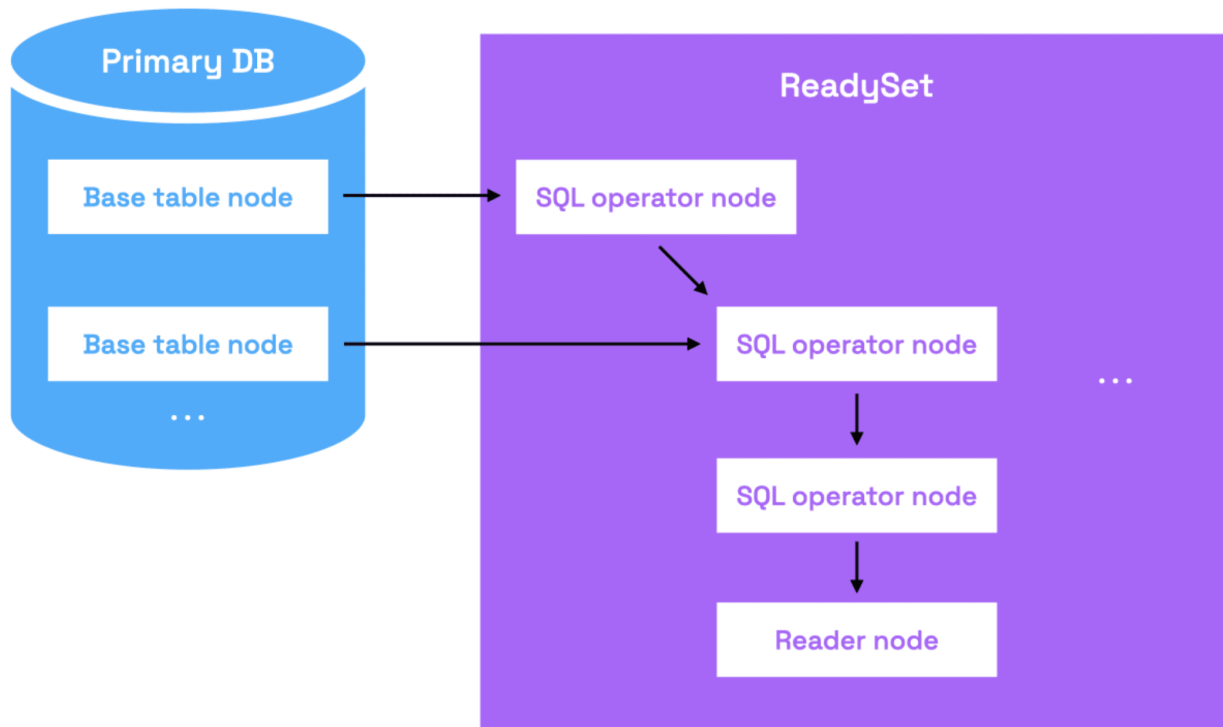


Deep Caching

- Snapshot
- Replication / CDC
- Proxy
- Dataflow
- Partial Materialization
- Cache Miss
- Incremental View Maintenance (IVM)



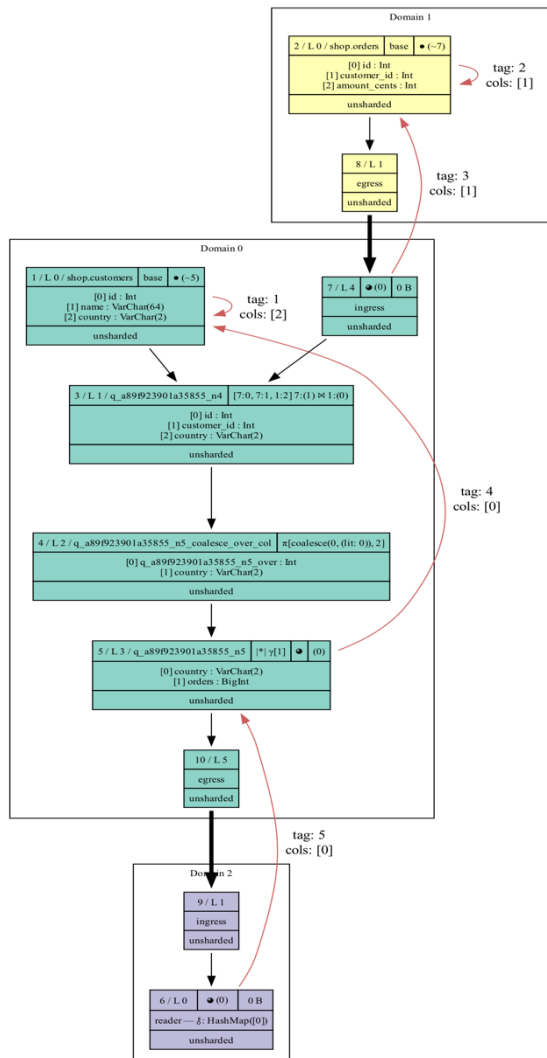
Deep Caching



Deep Caching

```
SELECT c.country, COUNT(*) AS orders
FROM orders o
INNER JOIN customers c ON o.customer_id = c.id
WHERE c.country = ?
GROUP BY c.country;
```

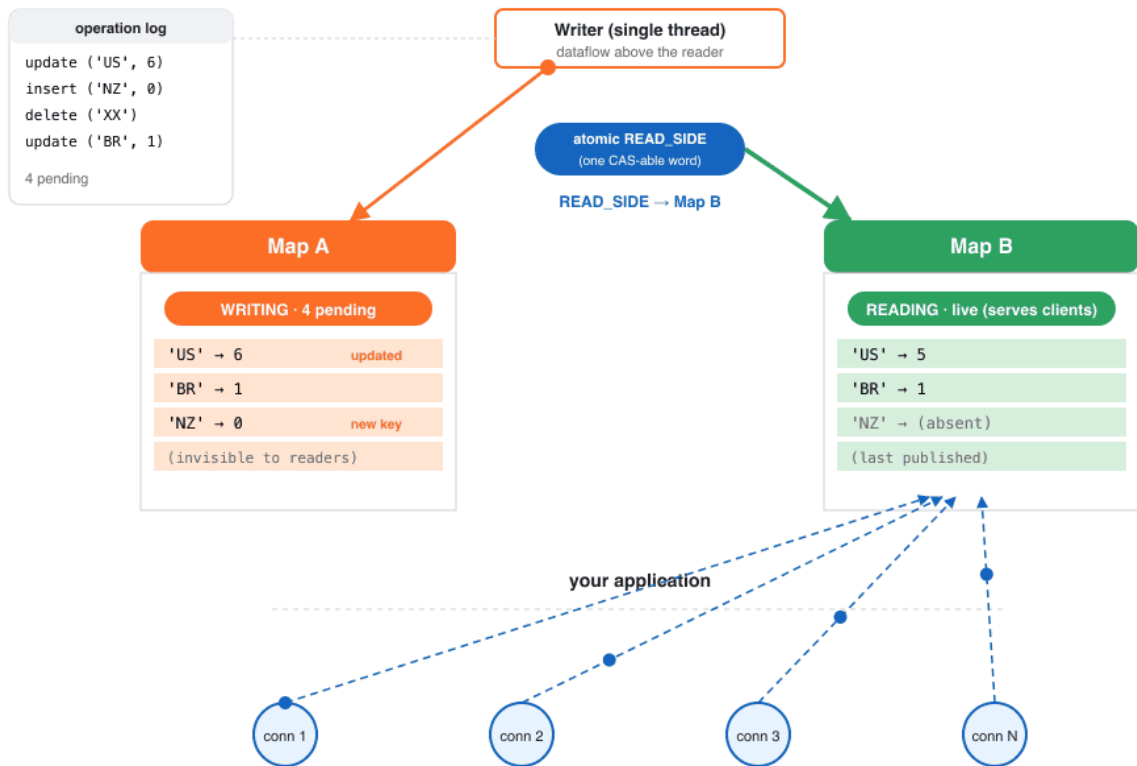
Deep Caching



Readysset reader: the left-right pattern

one writer · two map copies · one atomic pointer · readers stay lock-free

PHASE 1 OF 4 · Writer applies ops to Map A · Readers serve from Map B



Demo



Database: **MySQL** ReadySet Instance: **Nano (1 vCPU / 0.5 GB)** Concurrency: **25** RS - upstream capped at 25 **Apply** **Start Workload** **Stop Workload** **Reset Stats** **STOPPED**

0/22 QUERIES CACHED | **0.0%** OF TRAFFIC SERVED FROM CACHE | **1** READ REPLICA AVOIDED

WITHOUT READYSET

MySQL · direct

Round-trip to the database on every read

AVG QUERY LATENCY

ms

THROUGHPUT

q/s

p99

ms

VS

AWAITING
WORKLOAD

WITH READYSET

Shallow cache · in-memory

Drop-in, wire-compatible, zero code changes

AVG QUERY LATENCY

ms

THROUGHPUT

q/s

p99

ms

LATENCY OVER TIME

1.0 ms

LAST -50 S

upstream p50 upstream p99 readysset p50 readysset p99

QPS OVER TIME

1.0 q/s

LAST -50 S

upstream readysset

Hide details

THROUGHPUT IMPROVEMENT

--

vs q/s

MYSQL AVG

0

ms

p50 p99

READYSET AVG

0

ms

p50 p99

MYSQL QPS

0

q/s

avg p50 p99

READYSET QPS

0

q/s

avg p50 p99

READ REPLICAS NEEDED

--

to match RS throughput

CACHED

0

queries

PROXIED

22

queries

TRAFFIC CACHED

0%

of Ziplian traffic

RS HIT RATE

100

% (since RS start)

RS MISSES

8

hits 636,267

RS HEAP

28.2

MB heap (eviction ticks: 3,853)

RS QUERIES SERVED

636,275

cumulative

CACHE REFRESHES

32

to upstream

ACTIVE CLIENTS

1

connections

UPSTREAM CONNS

0

RS → upstream

DISTINCT SHAPES

40

query patterns

RS RESIDENT

118.8

MB (RSS)

- your turn

Try Readysset.

Put your most painful query behind the proxy.
Same driver, no application rewrite.
Measure the before and the after.

01 → `docker run` · spin it up locally in minutes

02 → PostgreSQL wire protocol · just change the DSN

03 → `auto-mode` · Readysset creates the caches automatically

point your camera



readysset.io

Questions?



• THE FUTURE RUNS ON MYSQL

MySQL BR Conf 2026

A full day of technical talks, real-world cases, and networking with the people who build, scale, and love the MySQL ecosystem in Brazil.



September 26, 2026 · Saturday



Anhembi Morumbi Mooca · São Paulo

Thank You!

